

Pageflow — Preserving the URL After a Request

Keeping the browser address bar unchanged after a Pageflow visit, across the PHP responder and the TypeScript UI router.

Plugin: `Plugins\Pageflow` · solves `pageflow.spa` · 2026-07-05

1. Overview

Pageflow is the framework's server-driven SPA layer (Inertia-style). By default, a visit updates the browser URL to match the page it renders. Sometimes you want to fire a request — activate a project, toggle a flag, submit a background action — **without** the address bar navigating away from the current page.

Pageflow supports this in **two** independent, fully-wired ways:

Approach	Where it lives	Use when
Server echo <code>loadPage: false</code> + <code>X-Pageflow-Url</code> / <code>X-Pageflow-Page</code> headers	Client sends headers; PHP responder echoes them back into the page object	Porting the legacy-framework pattern; you want the server to confirm the URL/component
Native client flag <code>preserveUrl: true</code>	Pure UI — history layer skips the URL write, no headers needed	New code; simplest way to freeze the URL

2. The Server-Echo Pattern (legacy-compatible)

Client call

```

const onActivate = (project) => {
  router.post("/ajax/v1/project/activate", { prid: project.$id }, {
    preserveScroll: true,
    preserveState: true,
    loadPage: false, // ← do not adopt the server's URL/component
    only: ["project", "isProjectError", "errorMessage", "errors"],
    headers: { // ← echo the current location back to PHP
      "X-Pageflow-Url": window.location.href,
      "X-Pageflow-Page": components,
    },
    replace: false,
    onSuccess: (res) => {
      if (res.props.isProjectError) {
        toast.error("Error activating project", { description: res.props.errorMessage });
      } else {
        window.location.reload();
      }
    },
    onError: () => toast.error("Failed to activate project."),
  });
};

```

✓ Header names are case-insensitive

Sending `x-pageflow-url` (lowercase) is fine — HTTP header names are case-insensitive and the kernel's `$request->header()` lookup matches `X-Pageflow-Url` regardless of case.

How the UI carries it

- Custom `headers` are spread verbatim into the request: `requestParams.ts:104` → `request.ts:135`.
- `loadPage` is a first-class `VisitOptions` flag (`types.ts:129`) threaded into `currentPage.set()` (`response.ts:164`, `page.ts:43`).
- `only` becomes the `X-Pageflow-Partial-Data` header for a partial reload (`requestParams.ts:114`).

How the PHP responder honours it

`plugins/Pageflow/Http/PageflowResponder.php:140–144`

```

$url = $this->fullUrl($request);
if (!$loadPage) {
    $headerUrl = (string) ($request->header('X-Pageflow-Url') ?? '');
    $parsed    = $headerUrl !== '' ? (parse_url($headerUrl, PHP_URL_PATH) ?: $url) : $url;
    $url       = (string) $parsed;
    $component = (string) ($request->header('X-Pageflow-Page') ?? $component);
}

$page = new PageflowPage(
    component: $component, // echoed from X-Pageflow-Page
    props:     $props,     // filtered by X-Pageflow-Partial-Data (only[])
    url:       $url,       // echoed from X-Pageflow-Url (path only)
    version:   $this->version,
);

```

The responder returns the client's current URL and component in the page object, so the browser's history entry is unchanged — the address bar stays put while the requested props are refreshed.

3. The Native `preserveUrl` Flag (recommended for new code)

A purely client-side alternative that needs no custom headers and no server cooperation:

```
router.post("/ajax/v1/project/activate", { prid: project.$id }, {
  preserveUrl: true, // ← freeze the address bar, client-side
  preserveScroll: true,
  preserveState: true,
  only: ["project", "isProjectError", "errorMessage", "errors"],
  onSuccess: (res) => { /* ... */ },
});
```

- `preserveUrl` is a real `VisitOptions` field (`types.ts:128`).
- When set, the response keeps `currentPage.get().url` instead of the server URL (`response.ts:159`).
- The history layer skips both `pushState` and `replaceState` URL writes (`history.ts:42, 152`).

4. Request Lifecycle

```
router.post(url, data, options)
    ↓
RequestParams → merges custom headers, sets X-Pageflow-Partial-Data from only[]
    ↓
Request.getHeaders() → adds X-Pageflow, version, CSRF (mutating + same-origin)
    ↓
PHP PageflowResponder → if !loadPage, echo X-Pageflow-Url / X-Pageflow-Page
    ↓
Response.setPage() → preserveUrl keeps current URL; loadPage passed to page.set()
    ↓
currentPage.set() → history push/replace (or skipped) → props swapped in
```

5. Option Reference

Option	Type	Effect
<code>preserveUrl</code>	boolean	Keep the current address-bar URL; skip history URL write entirely (client-only).
<code>loadPage</code>	boolean	When <code>false</code> , do not adopt the server's URL/component — pair with the echo headers.
<code>preserveScroll</code>	boolean	Do not reset scroll position after the visit.
<code>preserveState</code>	boolean	Keep local component state across the prop swap.
<code>only</code>	string[]	Partial reload — request only these props (<code>X-Pageflow-Partial-Data</code>).
<code>replace</code>	boolean	Use <code>history.replaceState</code> instead of <code>pushState</code> .
<code>headers</code>	object	Extra request headers, merged verbatim — carries <code>X-Pageflow-Url</code> / <code>X-Pageflow-Page</code> .

6. Which One Should I Use?

Rule of thumb

Reach for `preserveUrl: true` in new code — it is the smallest, header-free way to freeze the URL. Use the `loadPage: false + X-Pageflow-Url / X-Pageflow-Page` header pattern when you are porting the legacy-framework behaviour or need the server to confirm the URL and component in the returned page object.

✓ Both are fully implemented

The PHP responder and the TypeScript router already support this end-to-end. Since the example also calls `window.location.reload()` on success, either approach produces identical user-visible behaviour.